VISUAL PROGRAMMING FOR DATA SCIENCE

PRACTICAL 1

Date Given	**: 10th MARCH 2026 (TUESDAY)**
Submission Date	**: 17th MARCH 2026 (TUESDAY)**
Date Submitted	**: 17th MARCH 2026 (TUESDAY)**
Weightage	**: 20%**
Semester	**: FEBRUARY 2026**
Lecturer	**: AHMAD LUQMAN BIN AHMAD KAMAL ARIFFIN**

Instruction to students:

- This is a Group Assignment.
- Complete this cover sheet and attach it to your Assignment (first page).

Student declaration:

We declare that:

- ***This assignment is our own work***
- ***We understand what is meant by plagiarism***
- ***Our lecturer has the right to deduct our marks in the case of:***
  - ***Late submission***

Any plagiarism found in our assignment.

MUHAMMAD HARISH HAQIM BIN ADNAN	***AFNAN FARISH BIN ABDUL RAHIM***	***KUVALESAN A/L RAVICHANDRAN***	***SURIANA BINTI ABDULLAH***
012025020434	***012025021256***	***012025021382***	***012025021263***

	Total	
MARKS:		**100**

ACKNOWLEDGEMENT

We the members of Group A, would like to express our heartfelt gratitude to our lecturer, Sir Ahmad Luqman Bin Ahmad Kamal Ariffin, for his invaluable guidance and encouragement throughout the course. The lecturer's explanation and support during the lectures have significantly improved the group's comprehension of the various data cleaning and transformation techniques. The lecturer's feedback has also been instrumental in shaping the quality of the work produced by the group members.

We also wish to express our gratitude to the entire group members, Afnan, Haqim, Kuva, and Suriana, for their collaboration and efforts in the production of the assignment. The entire group members were instrumental in the production of the assignment. The spirit of cooperation and the entire group members' efforts ensured that the group was able to successfully complete the practical.

TABLE OF CONTENTS

1.0 INTRODUCTION 1
1.1 Objective 1
1.2 Scope of Assignment 1
1.3 Significance 1
2.0 DATA IMPORT AND BASIC INSPECTION 2
2.1 Import Pandas Library 2
2.2 Load The Raw Dataset 3
2.3 Data Types 3
2.4 The Original Shape of the Dataset 4
2.5 The Head Function 4
2.6 The Dataset Information 5
2.7 Total Null Values from Each Column 5
3.0 HANDLING DUPLICATES AND MISSING VALUES 7
3.1 Count Duplicates 7
3.2 Display Duplicate Rows 7
3.3 Dropping Duplicate Rows 9
3.4 Calculating the Age Mean 10
3.5 Fill Missing Age with Mean Value 10
3.6 Finding The Product Mode 11
3.7 Fill Missing Product Category Data with Mode Value 11
3.8 Dropping Rows with Missing Customer Name 12
4.0 HANDLING OUTLIERS 13
4.1 Q1 and Q3 Calculation 13
4.2 IQR Calculation 13
4.3 Calculations of Lower and Upper Bound 14
4.4 Identify the Outliers Rows 14
4.5 Filtering the Outliers 15
4.6 New Shape of The Dataset 16

5.0 DATA TRANSFORMATION AND EXPORT 17
5.1 New Column: Spend_Category 17
5.2 Standardized Product_Category Column 19
5.3 Save Fully Cleaned and Transformed Dataset 20
6.0 PRESENTATION 21
7.0 CONCLUSION 24
REFERENCES 25

LIST OF FIGURES

Figure 2.1: Program code for importing Panda's library 2
Figure 2.2: Program code and output for load raw dataset 3
Figure 2.3: Program code and output for the datatypes 3
Figure 2.4: Program code and output for original shape of the dataset 4
Figure 2.5: Program code and output for head function 4
Figure 2.6: Program code and output for the dataset's information 5
Figure 2.7: Program code and output for total null values 5
Figure 3.1: Program code and output for counting duplicates 7
Figure 3.2: Program code and output for displaying duplicate rows 7
Figure 3.3: Program code and output for displaying the exact duplicated rows 8
Figure 3.4: Program code and output for dropping duplicate rows 9
Figure 3.5: Program code and output for age mean 10
Figure 3.6: Program code and output for filling missing age 10
Figure 3.7: Program code and output for finding the product mode 11
Figure 3.8: Program code and output for filling missing product category 11
Figure 3.9: Program code and output for dropping rows with missing customer name 12
Figure 4.1: Program code and output for calculating Q1 and Q3 13
Figure 4.2: Program code and output for calculating IQR 13
Figure 4.3: Program code and output for calculating the bounds 14
Figure 4.4: Program and output for displaying outliers 14
Figure 4.5: Program code and output for filtering the outliers 15
Figure 4.6: Program code and output for printing the new shape 16
Figure 5.1: Program code and output for new column and categorise the data 17
Figure 5.2: Program code and output for standardized product category 19

Figure 5.3: Program code and output for saving the new dataset 20
Figure 6.1: Slide 1 (Initial State) 21
Figure 6.2: Slide 2 (Initial State – Continue) 22
Figure 6.3: Slide 3 (Cleaning Process – Missing Data and Duplicates Rows) 22
Figure 6.4: Slide 4 (Cleaning Process – Outliers Data) 23
Figure 6.5: Slide 5 (Data Transformation and Export) 23

EXTERNAL LINKS FOR PROGRAM CODES, DATASETS SLIDES PRESENTATION, AND SOFTCOPY REPORT

1. Google Drive Link: https://drive.google.com/drive/folders/1B_cn9YNqoTEEo-gHZ8H_yiTJ5oCT1hCC?usp=sharing
2. GitHub Link: https://github.com/HaqimSaintis/VPDS-Lab-Practical-1.git

CANVA LINK FOR SLIDES PRESENTATION

1. Canva Link: https://www.canva.com/design/DAHDsMPysjA/IrCWNgsJYuw3YW22Ao3aIw/edit?utm_content=DAHDsMPysjA&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

1.0 INTRODUCTION

This lab practical 1, serves as a foundational exercise in the course CCS23503 Visual Programming for Data Science. We use data science tools such as Python programming language with Pandas library within a Jupyter Notebook from Anaconda Navigator software for this assignment.

1.1 Objective

The primary objective of this lab practical 1 is to perform data cleaning, transformation, and integration techniques using visual programming tools (GeeksForGeeks, 2026). Data in its raw form is rarely ready for immediate analysis where it is often plagued by "dirty" data issues such as missing values, duplicate records, and extreme outliers that can distort business reports and lead to inaccurate conclusions. By engaging with the provided P1_data.xlsx dataset, we identify these quality issues and apply programmatic solutions to resolve them.

1.2 Scope of Assignment

This practical application is divided into several critical phases of the data science lifecycle such as:

a) Data inspection: Using the panda's library to load data and identify the exact count of missing values and data types (GeeksForGeeks, 2026).
b) Data cleaning: Implementing advanced techniques such as mean imputation for numerical gaps (Age), mode imputation for categorical blanks (Product_Category), and the removal of duplicate records (GeeksForGeeks, 2026).
c) Statistical filtering: Utilizing the Interquartile Range (IQR) method to detect and remove extreme anomalies in spending data, ensuring the integrity of the final analysis (GeeksForGeeks, 2026).
d) Data transformation: Transforming the cleaned data by creating new categorical features, such as a Spend_Category (Low, Medium, High), and standardizing text formats to uppercase for consistency (GeeksForGeeks, 2026).

1.3 Significance

By completing this assignment, we will demonstrate our proficiency in maintaining a logical and chronological data workflow from raw import to the export of a refined final_P1_data.csv file.

2.0 DATA IMPORT AND BASIC INSPECTION

According to (Coursera Staff, 2025), Pandas is an open-source library in Python for data analysis and manipulation. It allows us to efficiently work with structured data through flexible data structures and various data-handling capabilities. To build our understanding of pandas and how we could use it for our professional tasks such as data inspection, data cleaning and data transformation for this assignment.

2.1 Import Pandas Library

Visual Programming for Data Science - CCS23503

Lab Practical 1

Group 1

Question 1

[1]:
```
# Import pandas library
import pandas as pd
pd.__version__ # Print pandas version
```

[1]:
```
'2.3.3'
```

Figure 2.1: Program code for importing Panda's library

Figure 2.1 shows the program code for importing Panda's library and output the current version of the Pandas library. The as pd is to create the alias name for our Pandas library. pd.__version__ is used to display the current version of our Pandas library installed inside the system.

2.2 Load The Raw Dataset

```
[2]: # Load and read raw data
     df = pd.read_excel('P1_data.xlsx')
     df
```

[2]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28.0	electronics	150.50
1	1002	Customer_2	34.0	Clothing	89.99
2	1003	Customer_3	45.0	HOME	210.00
3	1004	Customer_4	NaN	electronics	340.50
4	1005	Customer_5	22.0	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39.0	electronics	330.00
76	1072	Customer_72	46.0	NaN	195.00
77	1073	Customer_73	32.0	beauty	85.00
78	1074	Customer_74	51.0	electronics	245.50
79	1075	Customer_75	27.0	Clothing	90.00

80 rows × 5 columns

Figure 2.2: Program code and output for load raw dataset

Figure 2.2 shows the program code for the load raw dataset alongside with the output. The df variable is initialised and assigned to the excel file name P1_data.xlsx by using the function read_excel. As we can see, the dataset is not fully cleaned, not fully transformed, and there is some missing values and null values.

2.3 Data Types

```
[3]: # Print data types of dataset's data
     print("The dataset's datatypes")
     df.dtypes

     The dataset's datatypes

[3]: Transaction_ID        int64
     Customer_Name        object
     Age                 float64
     Product_Category     object
     Total_Spend         float64
     dtype: object
```

Figure 2.3: Program code and output for the datatypes

Figure 2.3 shows the program code for the dataset's datatypes along with its output. The function .dtypes is used to display all data types from each column of the dataset. There are over three different data types from the dataset which are int64, object, and float64.

2.4 The Original Shape of the Dataset

```
[4]: # Print the new shape of the dataset
     print(f'''The original shape of the dataset is {df.shape},
     where {df.shape[0]} is the number of rows and {df.shape[1]} is the
     number of columns.''')

     The original shape of the dataset is (80, 5),
     where 80 is the number of rows and 5 is the
     number of columns.
```

Figure 2.4: Program code and output for original shape of the dataset

Figure 2.4 shows the program code and output for original shape of the dataset where 80 is the number of rows and 5 is the number of columns.

2.5 The Head Function

```
[5]: # Print the top five rows from the dataset
     df.head()
```

[5]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28.0	electronics	150.50
1	1002	Customer_2	34.0	Clothing	89.99
2	1003	Customer_3	45.0	HOME	210.00
3	1004	Customer_4	NaN	electronics	340.50
4	1005	Customer_5	22.0	beauty	45.00

Figure 2.5: Program code and output for head function

Figure 2.5 shows the program code and output for the head function of the data frame. The .head() function is used to display the top 5 rows of the dataset. It displays the top five rows from the dataset.

2.6 The Dataset Information

```
[6]: # Print the dataset's information
     print("The dataset's information")
     df.info()

     The dataset's information
     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 80 entries, 0 to 79
     Data columns (total 5 columns):
      #   Column            Non-Null Count  Dtype
     ---  ------            --------------  -----
      0   Transaction_ID    80 non-null     int64
      1   Customer_Name     76 non-null     object
      2   Age               73 non-null     float64
      3   Product_Category  73 non-null     object
      4   Total_Spend       80 non-null     float64
     dtypes: float64(2), int64(1), object(2)
     memory usage: 3.3+ KB
```

Figure 2.6: Program code and output for the dataset's information

Figure 2.6 shows the program code for the dataset information. The .info() function outputs the dataset information consist of the column names, the index, the non-null count and the datatype of each column from the dataset. Information that was included such as the type class of the df variable, the total columns, the column name, total non-null count and the data types of each column.

2.7 Total Null Values from Each Column

```
[7]: # Print the total null values from each column
     print('Total null values from each column:')
     df.isnull().sum()

     Total null values from each column:

[7]: Transaction_ID      0
     Customer_Name       4
     Age                 7
     Product_Category    7
     Total_Spend         0
     dtype: int64

     The columns that containing missing data are:
     1. Customer_Name
     2. Age
     3. Product_Category
```

Figure 2.7: Program code and output for total null values

Figure 2.7 shows the program code for the total null values from each column. It shows how many null values were found in each column by using .isnull().sum(). There are only three column that contain null values. The explanation about the columns containing missing data shown in the text cell below from the output Figure 2.6. The Transcation_ID column contains 0 null values, the Customer_Name column contains 4 null values, the Age column contains 7 null values, the Product_Category column contains 7 null values and finally the Total_Spend contains 0 null values.

3.0 HANDLING DUPLICATES AND MISSING VALUES

In this section, we will discuss and code Python on how to handle the duplicates and missing values from the dataset. We will count how many duplicates, identify the duplicates rows, delete the duplicates rows and filling missing age values with mean value, filling missing product category values with the mode of the product category and finally delete rows that contains missing customer names.

3.1 Count Duplicates

Question 2

```
[8]: # Count duplicates
     print(f'Total duplicates: {df.duplicated().sum()}')

     Total duplicates: 5
```

Figure 3.1: Program code and output for counting duplicates

Figure 3.1 shows the program code for counting duplicates alongside with the output. It is stated that the total duplicates are five. The .duplicated() function is used to capture the duplicated rows and .sum() function is used to count the overall duplicated rows.

3.2 Display Duplicate Rows

```
[9]: # Show the duplicate rows
     duplicate_rows = df[df.duplicated(keep=False)]
     duplicate_rows
```

[9]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
1	1002	Customer_2	34.0	Clothing	89.99
8	1009	Customer_9	41.0	HOME	180.20
13	1014	Customer_14	31.0	electronics	410.00
16	1002	Customer_2	34.0	Clothing	89.99
26	1026	Customer_26	44.0	Clothing	105.00
28	1009	Customer_9	41.0	HOME	180.20
42	1026	Customer_26	44.0	Clothing	105.00
53	1051	Customer_51	26.0	Clothing	85.00
57	1014	Customer_14	31.0	electronics	410.00
70	1051	Customer_51	26.0	Clothing	85.00

Figure 3.2: Program code and output for displaying duplicate rows

Figure 3.2 shows the program code for displaying duplicate rows alongside with the output. It is stated that there are over 10 rows that have the same rows information. If we want to display the exact 5 rows that are duplicated, we can change the “keep” parameter value into ‘first’ just like in Figure 3.3.

```
[10]: # Display five exact rows that have duplicated values
      df[df.duplicated(keep='first')]
```

[10]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
16	1002	Customer_2	34.0	Clothing	89.99
28	1009	Customer_9	41.0	HOME	180.20
42	1026	Customer_26	44.0	Clothing	105.00
57	1014	Customer_14	31.0	electronics	410.00
70	1051	Customer_51	26.0	Clothing	85.00

Figure 3.3: Program code and output for displaying the exact duplicated rows

Figure 3.3 shows the program code and output for displaying the exact duplicated rows that have the duplicated values.

3.3 Dropping Duplicate Rows

```
[11]: # Drop duplicates and save to df variable
      df = df.drop_duplicates(keep='first', inplace=False).copy()
      df
```

[11]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28.0	electronics	150.50
1	1002	Customer_2	34.0	Clothing	89.99
2	1003	Customer_3	45.0	HOME	210.00
3	1004	Customer_4	NaN	electronics	340.50
4	1005	Customer_5	22.0	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39.0	electronics	330.00
76	1072	Customer_72	46.0	NaN	195.00
77	1073	Customer_73	32.0	beauty	85.00
78	1074	Customer_74	51.0	electronics	245.50
79	1075	Customer_75	27.0	Clothing	90.00

75 rows × 5 columns

Figure 3.4: Program code and output for dropping duplicate rows

Figure 3.4 shows the program code and output for duplicate rows. The code drops only five rows and as a result the dataset now have 75 rows out of 80 rows from the old dataset. Our goal is to remove the identified duplicated rows. To delete duplicated rows, we used the drop_duplicates() function follow with the “keep” parameter that has ‘first’ as its value and the inplace and .copy() function to make sure that the df variable don’t change the original dataset. Since this operation requires changes to the dataset, we made sure to assign the variable to the default name of the dataset which is “df”. The output in Figure 3.4 above shows the first page of the dataset without the duplicate rows.

3.4 Calculating the Age Mean

```
[12]: # Calculate the age mean
      age_mean = round(df['Age'].mean())
      print(f'The age mean is {age_mean}')

      The age mean is 37
```

Figure 3.5: Program code and output for age mean

In Figure 3.5 above, we wrote a code to calculate the mean age for the dataset. We assigned the operation with a new variable as it doesn't make any changes to the dataset. To calculate the mean, we used the mean() function with the addition of round() function because we wanted to make sure the output is a round number since it involves age.

3.5 Fill Missing Age with Mean Value

```
[13]: # Fill missing Age with mean value
      df['Age'] = df['Age'].fillna(age_mean).astype(int)
      df
```

[13]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28	electronics	150.50
1	1002	Customer_2	34	Clothing	89.99
2	1003	Customer_3	45	HOME	210.00
3	1004	Customer_4	37	electronics	340.50
4	1005	Customer_5	22	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00
76	1072	Customer_72	46	NaN	195.00
77	1073	Customer_73	32	beauty	85.00
78	1074	Customer_74	51	electronics	245.50
79	1075	Customer_75	27	Clothing	90.00

75 rows × 5 columns

Figure 3.6: Program code and output for filling missing age

Figure 3.6 shows the program code and output for filling missing age with mean value. The code calculates the sum of all ages and divided with 75. The result from the calculation will fill up the missing age with the mean values.

3.6 Finding The Product Mode

```
[14]: # Find the mode from product category column
      product_mode = df['Product_Category'].mode()[0]
      print(f'The product mode is "{product_mode}"')

      The product mode is "electronics"
```

Figure 3.7: Program code and output for finding the product mode

Figure 3.7 shows the program code and output for finding the mode from the Product_Category column. It is stated that the mode is “electronics”.

3.7 Fill Missing Product Category Data with Mode Value

```
[15]: # Fill missing product category with mode value
      df['Product_Category'] = df['Product_Category'].fillna(product_mode)
      df
```

[15]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28	electronics	150.50
1	1002	Customer_2	34	Clothing	89.99
2	1003	Customer_3	45	HOME	210.00
3	1004	Customer_4	37	electronics	340.50
4	1005	Customer_5	22	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00
76	1072	Customer_72	46	electronics	195.00
77	1073	Customer_73	32	beauty	85.00
78	1074	Customer_74	51	electronics	245.50
79	1075	Customer_75	27	Clothing	90.00

75 rows × 5 columns

Figure 3.8: Program code and output for filling missing product category

Figure 3.8 shows the program code and output for filling missing product category with mode value. It filled up all the empty or missing data with mode value which is the “electronics” by using the function .fillna(). Like the missing data in the age column, we made sure to write a code that made sure that it filled in only the Product_Category column.

3.8 Dropping Rows with Missing Customer Name

```
[16]: # Drop rows where Customer_Name is missing
      df = df.dropna(subset=['Customer_Name'], inplace=False).copy()
      df
```

[16]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28	electronics	150.50
1	1002	Customer_2	34	Clothing	89.99
2	1003	Customer_3	45	HOME	210.00
3	1004	Customer_4	37	electronics	340.50
4	1005	Customer_5	22	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00
76	1072	Customer_72	46	electronics	195.00
77	1073	Customer_73	32	beauty	85.00
78	1074	Customer_74	51	electronics	245.50
79	1075	Customer_75	27	Clothing	90.00

71 rows × 5 columns

Figure 3.9: Program code and output for dropping rows with missing customer name

Figure 3.9 shows the program code and output for dropping rows with missing customer name. we used the dropna() function with the addition of subset=() condition to make sure that it checks the Customer_Name column for the missing value. The dropna() function will then “drop” the rows that contains missing customer name. Only 4 rows have been dropped and as a result there are only 71 rows were left in the dataset.

4.0 HANDLING OUTLIERS

In this section, we will discuss and code Python on how to handle the outliers from the dataset. We will first calculate the quartiles and inter quartile range to find the value of the upper bound and lower bound and finally we will the delete the outliers by using the bounds that we have calculated.

4.1 Q1 and Q3 Calculation

```
[17]: # Calculate the Q1 and Q3
      Q1 = df['Total_Spend'].quantile(0.25)
      Q3 = df['Total_Spend'].quantile(0.75)
      # Display the value of Q1 and Q3
      print(f'The value of Q1 is {Q1}')
      print(f'The value of Q3 is {Q3}')

      The value of Q1 is 92.75
      The value of Q3 is 277.5
```

Figure 4.1: Program code and output for calculating Q1 and Q3

Figure 4.1 shows the program code and output for calculating first quartile (Q1) and third quartile (Q3). We use the ".quantile" function to calculate both quartile and display it using "f" format print.

4.2 IQR Calculation

```
[18]: # Calculate the IQR
      IQR = Q3 - Q1
      # Display the value of IQR
      print(f'The value of IQR is {IQR}')

      The value of IQR is 184.75
```

Figure 4.2: Program code and output for calculating IQR

Figure 4.2 shows the program code and output for calculating the IQR (Interquartile Range) value. We subtract it by Q3 - Q1 and get the result for IQR.

4.3 Calculations of Lower and Upper Bound

```
[19]: # Define and calculate the bounds
      lower_bound = Q1 - (1.5 * IQR)
      upper_bound = Q3 + (1.5 * IQR)
      # Display the lower bound and upper bound
      print(f'The value of lower bound: {lower_bound}')
      print(f'The value of upper bound: {upper_bound}')

      The value of lower bound: -184.375
      The value of upper bound: 554.625
```

Figure 4.3: Program code and output for calculating the bounds

Figure 4.3 shows the program code and output for calculating the lower bound and upper bound. The lower bound is calculated by Q1 subtracting with 1.5 times with the IQR. Like upper bound where it is calculated by Q3 addition with 1.5 times with the IQR. Finally, we display the result for those bounds.

4.4 Identify the Outliers Rows

```
[20]: # Create a filter for the outliers (values outside the bounds)
      outliers_df = df[(df['Total_Spend'] < lower_bound) | (df['Total_Spend'] > upper_bound)]

      # Display the outlier rows
      print("Detected outlier rows:")
      print(outliers_df)

      Detected outlier rows:
          Transaction_ID Customer_Name  Age Product_Category  Total_Spend
      7             1008    Customer_8   29      electronics       3500.0
      18            1018   Customer_18   42             HOME       4200.5
      33            1032   Customer_32   49      electronics       2800.0
      46            1044   Customer_44   22      electronics       5500.0
      65            1062   Customer_62   54             HOME       4100.0
```

Figure 4.4: Program and output for displaying outliers

Figure 4.4 shows the program code and output for displaying the outlier's data. There are 5 rows that have outlier data. The operation begins by creating a new DataFrame called outliers_df, which acts as a temporary container to store these problematic rows without deleting them from the main dataset immediately. Inside the square brackets, the code evaluates two separate conditions: the first checks if a value is less than the lower_bound, and the second checks if it exceeds the upper_bound. The use of the pipe symbol (|) is critical here as it represents the "OR" logic, meaning a row will be flagged if it meets either criteria of being an extreme low or an extreme high. This is essential because, mathematically, a single

transaction cannot be both below the minimum and above the maximum simultaneously. Finally, the print function outputs the resulting rows.

4.5 Filtering the Outliers

```
[21]: # Filter the outliers data
df = df[(df['Total_Spend'] >= lower_bound) & (df['Total_Spend'] <= upper_bound)]
df
```

[21]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28	electronics	150.50
1	1002	Customer_2	34	Clothing	89.99
2	1003	Customer_3	45	HOME	210.00
3	1004	Customer_4	37	electronics	340.50
4	1005	Customer_5	22	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00
76	1072	Customer_72	46	electronics	195.00
77	1073	Customer_73	32	beauty	85.00
78	1074	Customer_74	51	electronics	245.50
79	1075	Customer_75	27	Clothing	90.00

66 rows × 5 columns

Figure 4.5: Program code and output for filtering the outliers

Figure 4.5 shows the program code and output for filtering the outliers from the dataset. This program removes outliers from a Data Frame using the Interquartile Range (IQR) method and removes 5 outliers from Total_Spend, shrinking the dataset from 71 to 66 rows. 5 rows were removed as they are identified as outliers and as a result there are 66 rows were left out of 71 rows.

a) df['Total_Spend'] >= lower_bound: This checks if the total spend is high enough to be considered "normal." Anything below this bound is a "low outlier" such as a negative value or an impossibly small number.
b) df['Total_Spend'] <= upper_bound: This checks if the total spend is low enough to be "normal." Anything above this is a "high outlier" such as 5500.00 or 4100.00 values seen in our raw data.
c) The & Operator: This is the logical AND where it requires a row to satisfy both conditions simultaneously to be kept.

When Pandas runs this code, it doesn't immediately delete rows. Instead, it creates a "mask" a list of True or False values for every row in our dataset. The summary from this code is that, if a row's spend is between the bounds, then it will mark as True. If a row is an outlier whether it is too high or too low, it will mark as False and removed from the data frame.

4.6 New Shape of The Dataset

```
[22]: # Print the new shape of the dataset
      print(f'''The new shape of the dataset is {df.shape},
      where {df.shape[0]} is the number of rows and {df.shape[1]} is the
      number of columns.''')

      The new shape of the dataset is (66, 5),
      where 66 is the number of rows and 5 is the
      number of columns.
```

Figure 4.6: Program code and output for printing the new shape

Figure 4.6 shows the program code and output for printing the new shape of the dataset. The new shape are 66 rows and 5 columns.

5.0 DATA TRANSFORMATION AND EXPORT

In this section, we will discuss and code Python on how to transform the data and transfer the new dataset into the new csv file. The dataset underwent a cleaning process which resulted in a transformation that enhanced its consistency while enabling future research work. The transformation process created a new categorical variable while text values from existing data were converted into standardized formats.

5.1 New Column: Spend_Category

Question 4

[23]:
```python
# Function to categorise the total spending
def categorise_total_spend(total_spend):
    if total_spend < 100:
        return 'Low'
    elif 100 <= total_spend <= 300:
        return 'Medium'
    else:
        return 'High'
# Copy the original dataset
df = df.copy()
# Apply the function to the new column in the dataset
df['Spend_Category'] = df['Total_Spend'].apply(categorise_total_spend)
# Display the current modified dataset
df
```

[23]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend	Spend_Category
0	1001	Customer_1	28	electronics	150.50	Medium
1	1002	Customer_2	34	Clothing	89.99	Low
2	1003	Customer_3	45	HOME	210.00	Medium
3	1004	Customer_4	37	electronics	340.50	High
4	1005	Customer_5	22	beauty	45.00	Low
...	...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00	High
76	1072	Customer_72	46	electronics	195.00	Medium
77	1073	Customer_73	32	beauty	85.00	Low
78	1074	Customer_74	51	electronics	245.50	Medium
79	1075	Customer_75	27	Clothing	90.00	Low

66 rows × 6 columns

Figure 5.1: Program code and output for new column and categorise the data

Figure 5.1 shows the program code and output for new column Spend_Category and categorise the data of total spend into different category. The data underwent transformation to improve both its usability and its consistency with the dataset which required cleaning. The system established a new column Spend_Category which it generated from the Total_Spend column data. The conditional function was used to divide spending into three distinct spending categories. The system identified transactions with total spending below 100 as Low while spending between 100 and 300 received Medium classification and spending above 300 received High classification.

1. The Logic Function (categorise_total_spend)

This is a conditional function that sorts the numerical values into the three distinct categories required by the rubric.

a) if total_spend < 100: Any customer who spent less than 100 is labeled as 'Low'.

b) elif 100 <= total_spend <= 300: Customers who spent between 100 and 300 (inclusive) are labeled 'Medium'.

c) else: Any value above 300 is automatically labeled 'High'.

2. Creating a Copy (df = df.copy())

a) This creates a "deep copy" of our DataFrame.

b) It is a safe programming practice that prevents the SettingWithCopyWarning we encountered earlier.

c) It ensures that the transformations we are about to make are applied to a fresh, independent version of the cleaned data.

3. Applying the Transformation (.apply)

a) df['Spend_Category']: This creates a brand-new column in our dataset.

b) .apply(categorise_total_spend): This tells pandas to take every single row in the Total_Spend column and run it through our function to determine the label.

4. Displaying the Result (df)

a) Simply typing df at the end displays the table so we can verify the new column exists and the values are correct.

5.2 Standardized Product_Category Column

[24]:
```
# Standardize the product category text to upper case
df['Product_Category'] = df['Product_Category'].str.upper()
df
```

[24]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend	Spend_Category
0	1001	Customer_1	28	ELECTRONICS	150.50	Medium
1	1002	Customer_2	34	CLOTHING	89.99	Low
2	1003	Customer_3	45	HOME	210.00	Medium
3	1004	Customer_4	37	ELECTRONICS	340.50	High
4	1005	Customer_5	22	BEAUTY	45.00	Low
...	...	...	...	...	...	...
75	1071	Customer_71	39	ELECTRONICS	330.00	High
76	1072	Customer_72	46	ELECTRONICS	195.00	Medium
77	1073	Customer_73	32	BEAUTY	85.00	Low
78	1074	Customer_74	51	ELECTRONICS	245.50	Medium
79	1075	Customer_75	27	CLOTHING	90.00	Low

66 rows × 6 columns

Figure 5.2: Program code and output for standardized product category

Figure 5.2 shows the program code and output for standardized Product_Category column. The str.upper() helps to make every rows of Product_Category column to upper case and finally the df variable is used to display the updated dataset. We standardized the Product_Category column by transforming all text values into uppercase letters. This method maintains consistent categorical values in the dataset because it prevents problems that arise from mixed text case usage.

5.3 Save Fully Cleaned and Transformed Dataset

[25]:
```
# Save the fully cleaned and transformed dataset as a new file
df.to_csv('final_P1_data.csv', index=False)
# Display the final dataset
df = pd.read_csv('final_P1_data.csv')
df
```

[25]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend	Spend_Category
0	1001	Customer_1	28	ELECTRONICS	150.50	Medium
1	1002	Customer_2	34	CLOTHING	89.99	Low
2	1003	Customer_3	45	HOME	210.00	Medium
3	1004	Customer_4	37	ELECTRONICS	340.50	High
4	1005	Customer_5	22	BEAUTY	45.00	Low
...	...	...	...	...	...	...
61	1071	Customer_71	39	ELECTRONICS	330.00	High
62	1072	Customer_72	46	ELECTRONICS	195.00	Medium
63	1073	Customer_73	32	BEAUTY	85.00	Low
64	1074	Customer_74	51	ELECTRONICS	245.50	Medium
65	1075	Customer_75	27	CLOTHING	90.00	Low

66 rows × 6 columns

End of the code

Figure 5.3: Program code and output for saving the new dataset

Figure 5.3 shows the program code and output for saving the new dataset to a new file name final_P1_data.csv. The to_csv function is used to save the dataset a new file with index=False. Using index=False tells pandas that "Save the data columns only and do not include the row numbers. Finally, the pd.read_csv function is used to display the current new dataset into the Jupyter Notebook. The cleaned data together with the transformed dataset was saved to a new csv file which received the name final_P1_data.csv. The final dataset contains 66 rows and 6 columns which includes the newly created spend_category column

6.0 PRESENTATION

In this section, we will make presentation slides and present to our lecturer where we will discuss about the initial state of the dataset, data cleaning process and the transformation of the dataset and export into a new file.

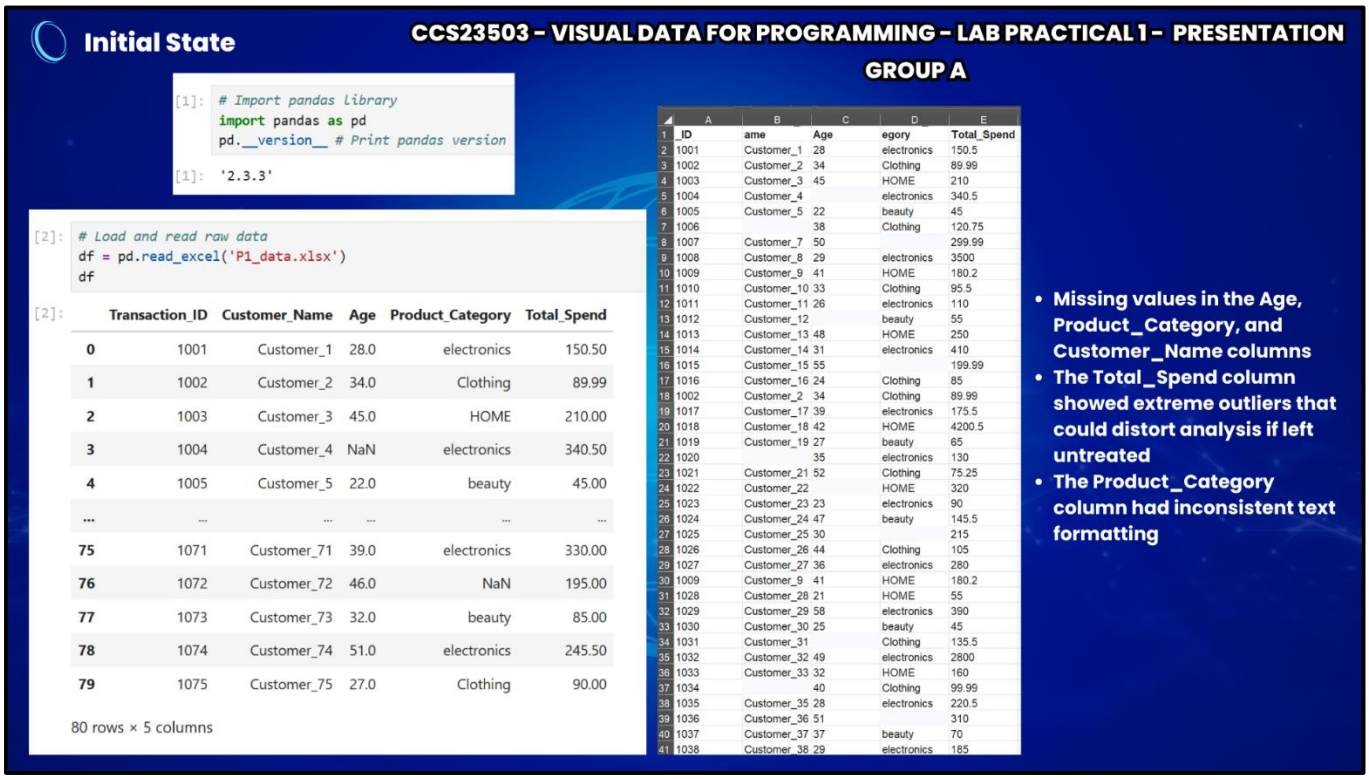

Figure 6.1: Slide 1 (Initial State)

Initial State (Continue)

```
[5]: # Print the top five rows from the dataset
     df.head()
```

[5]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28.0	electronics	150.50
1	1002	Customer_2	34.0	Clothing	89.99
2	1003	Customer_3	45.0	HOME	210.00
3	1004	Customer_4	NaN	electronics	340.50
4	1005	Customer_5	22.0	beauty	45.00

```
[6]: # Print the dataset's information
     print("The dataset's information")
     df.info()

     The dataset's information
     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 80 entries, 0 to 79
     Data columns (total 5 columns):
      #   Column            Non-Null Count  Dtype
     ---  ------            --------------  -----
      0   Transaction_ID    80 non-null     int64
      1   Customer_Name     76 non-null     object
      2   Age               73 non-null     float64
      3   Product_Category  73 non-null     object
      4   Total_Spend       80 non-null     float64
     dtypes: float64(2), int64(1), object(2)
     memory usage: 3.3+ KB
```

```
[7]: # Print the total null values from each column
     print('Total null values from each column:')
     df.isnull().sum()

     Total null values from each column:
[7]: Transaction_ID      0
     Customer_Name       4
     Age                 7
     Product_Category    7
     Total_Spend         0
     dtype: int64

     The columns that containing missing data are:
     1. Customer_Name
     2. Age
     3. Product_Category
```

Figure 6.2: Slide 2 (Initial State – Continue)

Cleaning Process: Missing Data and Duplicates Rows

```
[11]: # Drop duplicates and save to df variable
      df = df.drop_duplicates(keep='first', inplace=False).copy()
      df
```

```
[12]: # Calculate the age mean
      age_mean = round(df['Age'].mean())
      print(f'The age mean is {age_mean}')

      The age mean is 37
```

```
[13]: # Fill missing Age with mean value
      df['Age'] = df['Age'].fillna(age_mean).astype(int)
      df
```

```
[14]: # Find the mode from product category column
      product_mode = df['Product_Category'].mode()[0]
      print(f'The product mode is "{product_mode}"')

      The product mode is "electronics"
```

```
[15]: # Fill missing product category with mode value
      df['Product_Category'] = df['Product_Category'].fillna(product_mode)
      df
```

```
[16]: # Drop rows where Customer_Name is missing
      df = df.dropna(subset=['Customer_Name'], inplace=False).copy()
      df
```

Figure 6.3: Slide 3 (Cleaning Process – Missing Data and Duplicates Rows)

Cleaning Process: Outliers Data

```
[17]: # Calculate the Q1 and Q3
Q1 = df['Total_Spend'].quantile(0.25)
Q3 = df['Total_Spend'].quantile(0.75)
# Display the value of Q1 and Q3
print(f'The value of Q1 is {Q1}')
print(f'The value of Q3 is {Q3}')

The value of Q1 is 92.75
The value of Q3 is 277.5
```

```
[18]: # Calculate the IQR
IQR = Q3 - Q1
# Display the value of IQR
print(f'The value of IQR is {IQR}')

The value of IQR is 184.75
```

```
[19]: # Define and calculate the bounds
lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)
# Display the lower bound and upper bound
print(f'The value of lower bound: {lower_bound}')
print(f'The value of upper bound: {upper_bound}')

The value of lower bound: -184.375
The value of upper bound: 554.625
```

```
[20]: # Create a filter for the outliers (values outside the bounds)
outliers_df = df[(df['Total_Spend'] < lower_bound) | (df['Total_Spend'] > upper_bound)]

# Display the outlier rows
print("Detected outlier rows:")
print(outliers_df)

Detected outlier rows:
    Transaction_ID Customer_Name  Age Product_Category  Total_Spend
7             1008    Customer_8   29      electronics       3500.0
18            1018   Customer_18   42             HOME       4200.5
33            1032   Customer_32   49      electronics       2800.0
46            1044   Customer_44   22      electronics       5500.0
65            1062   Customer_62   54             HOME       4100.0
```

```
[21]: # Filter the outliers data
df = df[(df['Total_Spend'] >= lower_bound) & (df['Total_Spend'] <= upper_bound)]
df
```

[21]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend
0	1001	Customer_1	28	electronics	150.50
1	1002	Customer_2	34	Clothing	89.99
2	1003	Customer_3	45	HOME	210.00
3	1004	Customer_4	37	electronics	340.50
4	1005	Customer_5	22	beauty	45.00
...	...	...	...	...	...
75	1071	Customer_71	39	electronics	330.00
76	1072	Customer_72	46	electronics	195.00
77	1073	Customer_73	32	beauty	85.00
78	1074	Customer_74	51	electronics	245.50
79	1075	Customer_75	27	Clothing	90.00

66 rows × 5 columns

Figure 6.4: Slide 4 (Cleaning Process – Outliers Data)

Data Transformation and Export

```
[23]: # Function to categorise the total spending
def categorise_total_spend(total_spend):
    if total_spend < 100:
        return 'Low'
    elif 100 <= total_spend <= 300:
        return 'Medium'
    else:
        return 'High'
# Copy the original dataset
df = df.copy()
# Apply the function to the new column in the dataset
df['Spend_Category'] = df['Total_Spend'].apply(categorise_total_spend)
# Display the current modified dataset
df
```

```
[24]: # Standardize the product category text to upper case
df['Product_Category'] = df['Product_Category'].str.upper()
df
```

```
[25]: # Save the fully cleaned and transformed dataset as a new file
df.to_csv('final_P1_data.csv', index=False)
# Display the final dataset
df = pd.read_csv('final_P1_data.csv')
df
```

[25]:

	Transaction_ID	Customer_Name	Age	Product_Category	Total_Spend	Spend_Category
0	1001	Customer_1	28	ELECTRONICS	150.50	Medium
1	1002	Customer_2	34	CLOTHING	89.99	Low
2	1003	Customer_3	45	HOME	210.00	Medium
3	1004	Customer_4	37	ELECTRONICS	340.50	High
4	1005	Customer_5	22	BEAUTY	45.00	Low
...	...	...	...	...	...	...
61	1071	Customer_71	39	ELECTRONICS	330.00	High
62	1072	Customer_72	46	ELECTRONICS	195.00	Medium
63	1073	Customer_73	32	BEAUTY	85.00	Low
64	1074	Customer_74	51	ELECTRONICS	245.50	Medium
65	1075	Customer_75	27	CLOTHING	90.00	Low

66 rows × 6 columns

End of the code

Transformation Steps

- Created a new column – Spend_Category
- Low (<100), Medium (100–300), High (>300) based on Total_Spend.
- Standardized Product_Category values
- Converted all text values to uppercase to ensure consistency.
- Exported the cleaned dataset
- The final dataset was saved as final_P1_data.csv.

Recommendation

- Validate data during entry to reduce missing values.
- Apply automated data cleaning to detect duplicates and outliers.
- Use standardized formats for categorical data to improve analysis accuracy.

Figure 6.5: Slide 5 (Data Transformation and Export)

7.0 CONCLUSION

In conclusion, this lab practical 1 effectively illustrated how to perform data cleaning, transformation and preparation with the help of Python and Pandas library. The first step involved getting the information about the dataset. Then, we are examining the dataset for problems where there were also missing values, duplicate records, and incorrect data types. Data cleaning techniques such as removing duplicate records, ignoring missing values by mean and mode replacement and filtering outliers using Interquartile Range (IQR) were performed on the dataset to improve its quality.

Once the data was cleaned, transformation of data happened so that we can prepare a structured dataset for analysis. Using the Total_Spend values, we created a variable called Spend_Category which divided customers into three categoriesLow, Medium, and High. Moreover, to standardize the Product_Category column all the text values were converted into uppercase.

Finally, the prepared dataset was exported as final_P1_data. csv. In the end we have 66 rows and 6 columns, it means dataset process is done successfully. This hands-on exercise contributed to building crucial abilities such as data preprocessing and highlighted the significance of thoroughly preparing the data ahead of conducting any analyses.

REFERENCES

Coursera Staff. (2025, April 25th). *What Is Pandas in Python?*. Coursera. https://www.coursera.org/articles/what-is-pandas-in-python?msockid=0431fb56b078694022edec46b1f66850

GeeksForGeeks. (2026, February 17th). *Data Cleaning*. https://www.geeksforgeeks.org/data-analysis/data-cleansing-introduction/

RUBRIC for Assignment

CLO1 – Perform data cleaning, transformation, and integration techniques using visual programming tools. (P4, PLO3)

COMPONENTS of CRITERIA/ WEIGHTAGE (%)	CRITERIA/ WEIGHTAGE (%)	LEVELS of ACHIEVEMENTS/ DESCRIPTORS					TOTAL SCORE 100 marks			
		Excellent (4)	**Good (3)**	**Average (2)**	**Needs Improvement (1)**	**Not Attempted (0)**	**1**	**2**	**3**	**4**
Content (40%)	**Identification of Data Quality Issues (15%)**	Identifies exactly 100% of missing values, duplicate records, and numerical outliers using calculated statistical bounds.	Identifies between 80% and 99% of missing values, duplicate records, and numerical outliers using calculated statistical bounds.	Identifies between 50% and 79% of missing values, duplicate records, and numerical outliers using visual estimations rather than exact statistical bounds.	Identifies fewer than 50% of missing values, duplicate records, and numerical outliers within the dataset.	Does not attempt to identify missing values, duplicate records, or numerical outliers.				
	Execution of Data Cleaning (15%)	Replaces missing numerical data using mean imputation, replaces categorical data using mode imputation, and deletes rows exclusively for missing unique identifiers.	Applies mean or mode imputation for most columns but leaves 1 to 2 null values untreated or deletes 1 to 2 incorrect rows.	Applies a single imputation method across all data types indiscriminately and retains multiple untreated null values.	Deletes entire columns instead of targeted rows and applies incorrect data types during the imputation process.	Does not apply any imputation methods or deletion functions to clean the dataset.				
	Execution of Data	Constructs a conditional	Constructs a conditional	Constructs a conditional	Writes the categorization	Does not write a categorization				

COMPONENTS of CRITERIA/ WEIGHTAGE (%)	CRITERIA/ WEIGHTAGE (%)	LEVELS of ACHIEVEMENTS/ DESCRIPTORS					TOTAL SCORE 100 marks			
		Excellent (4)	**Good (3)**	**Average (2)**	**Needs Improvement (1)**	**Not Attempted (0)**	**1**	**2**	**3**	**4**
	Transformation (10%)	function to classify numerical spending into exactly three distinct categories and converts all targeted text entries to uppercase.	function to classify numerical spending into three categories but omits the uppercase conversion for 1 to 2 text entries.	function for categorization but leaves the targeted text entries entirely in their original mixed case format.	function with fatal syntax errors preventing execution and leaves the text format unmodified.	function and does not modify the text case format.				
Process (30%)	**Code Execution and Library Utilization (10%)**	Executes built-in pandas library functions to perform all data manipulation tasks without producing any runtime errors or relying on manual iterative loops.	Executes built-in pandas library functions for the majority of tasks but writes 1 to 2 manual iterative loops resulting in minor recoverable errors.	Executes manual iterative loops for the majority of tasks and requires multiple manual code restarts to process the data completely.	Executes standard Python text functions instead of pandas data manipulation functions resulting in fatal data type errors.	Does not write executable Python code and leaves the programming cells completely blank.				
	Data Import and Workspace Setup (10%)	Loads the designated CSV file into the Google Colab environment using the pandas library without requiring any manual file	Loads the designated CSV file into the Google Colab environment but requires 1 to 2 manual code adjustments to the file path to	Loads the designated CSV file into a local offline environment instead of utilizing the required Google Colab workspace.	Writes the pandas import statement but fails to load the designated CSV file into the programming environment.	Does not write the pandas import statement and does not load the designated CSV file.				

COMPONENTS of CRITERIA/ WEIGHTAGE (%)	CRITERIA/ WEIGHTAGE (%)	LEVELS of ACHIEVEMENTS/ DESCRIPTORS					TOTAL SCORE 100 marks			
		Excellent (4)	**Good (3)**	**Average (2)**	**Needs Improvement (1)**	**Not Attempted (0)**	**1**	**2**	**3**	**4**
		path adjustments.	execute successfully.							
	Workflow Sequencing and Logic (10%)	Sequences the operational steps in chronological order from raw data import to final data export without placing any cleaning tasks after the transformation phase.	Sequences the operational steps chronologically for the most part but places exactly 1 cleaning task incorrectly after the transformation phase.	Sequences the operational steps out of order requiring the user to run notebook code blocks non-linearly to achieve the final output.	Scrambles the operational steps completely causing fatal data dependency errors during code execution.	Does not arrange the operational steps into any sequence and submits unorganized code blocks.				
Product (30%)	**Completeness of the Cleaned Dataset (15%)**	Outputs a final exported CSV file containing exactly 0 duplicate records, 0 null values, and the newly categorized spending column without the programming index.	Outputs a final exported CSV file containing 0 duplicate records but retains 1 to 2 null values or accidentally includes the programming index.	Outputs a final exported CSV file containing 1 to 2 duplicate records and omits the required categorized spending column entirely.	Outputs a corrupted or empty final CSV file that retains 3 or more duplicate records and multiple null values.	Does not output a final dataset and does not generate an exported file.				
	Written Summary and Communicati	States the starting row count, lists the	States the starting row count and lists the applied	States the starting row count but omits both the	Omits the starting row count, the applied functions,	Does not submit a written summary document to				

COMPONENTS of CRITERIA/ WEIGHTAGE (%)	CRITERIA/ WEIGHTAGE (%)	LEVELS of ACHIEVEMENTS/ DESCRIPTORS					TOTAL SCORE 100 marks			
		Excellent (4)	**Good (3)**	**Average (2)**	**Needs Improvement (1)**	**Not Attempted (0)**	**1**	**2**	**3**	**4**
	on of Findings (10%)	exact cleaning and transformation functions applied, and states the final row count in the written summary document.	functions but omits the final row count in the written summary document.	applied functions and the final row count in the written summary document.	and the final row count resulting in an incomplete written summary document.	communicate findings.				
						OVERALL SCORE				